

Corrections de sécurité

Suite au test d'intrusion GSNA

Les 11 vulnérabilités identifiées — expliquées simplement, corrigées, et vérifiées par des tests.

Rapport source : **SquareMeter_Rapport_Test_Intrusion_VF_FR.md**
Résumé : **SECURITY_FIXES.md** • Version française enrichie : **SECURITY_FIXES_FR.pdf**
Date : 11 août 2026 • 47 tests de régression sécurité exécutés : **47 réussis, 0 échec**

1. Vue d'ensemble

Un audit de sécurité (test d'intrusion) a été réalisé sur le site **squaremeter.ma** par **GSNA Solutions**. **11 vulnérabilités** ont été découvertes, de gravité **Critique** à **Faible**. Toutes les vulnérabilités corrigeables dans le code ont été corrigées dans le commit `f73dd88 – "fix: remediate all 11 penetration test vulnerabilities"`.

ID	Vulnérabilité	Gravité	Correctif
V1	Identifiants API Apimo codés en dur dans le code client	Haute	Identifiants déplacés côté serveur (variables d'environnement)
V2	Proxy API ouvert sans authentification (CORS <code>*</code>)	Critique	CORS restreint aux domaines officiels + lecture seule (GET)
V3	Mot de passe « Espace Clients » codé en dur	Moyenne	Vérification déplacée côté serveur + anti force-brute
V4	Mot de passe section Mag codé en dur	Faible	Composant inutilisé supprimé
V5	Mot de passe du tableau de bord analytique codé en dur	Critique	Vérification déplacée côté serveur + anti force-brute
V6	Clé IA Gemini exposée au navigateur	Moyenne	Clé utilisée uniquement côté serveur (proxy <code>/api/chatbot</code>)
V7	XSS stocké via <code>dangerouslySetInnerHTML</code>	Haute	Assainissement HTML avec DOMPurify (liste blanche de balises)
V8	Paramètre <code>path</code> non assaini dans le proxy WordPress	Moyenne	Liste blanche de chemins (posts/pages/categories/tags)
V9	Injection de prompt dans le chatbot IA	Moyenne	Prompts durcis + assainissement des entrées utilisateur
V10	Données personnelles exposées via le WHOIS public	Faible	Correctif DNS / registre (hors code)
V11	Absence de SPF / DKIM / DMARC (usurpation e-mail)	Moyenne	Correctif DNS / messagerie (hors code)

RÉSULTAT

9 vulnérabilités corrigées dans le code (V1 à V9) + 2 correctifs d'infrastructure recommandés (V10, V11), plus des améliorations de sécurité supplémentaires (en-têtes HTTP, durcissement CORS, suppression d'un endpoint de debug).

2. Comment les correctifs sont vérifiés (tests)

Une suite de 47 tests de régression sécurité a été ajoutée au projet : scripts/security-regression-tests.js . Elle s'exécute sans aucune dépendance externe :

```
node scripts/security-regression-tests.js
```

RÉSULTAT : 47 tests réussis, 0 échec

Ces tests couvrent les 6 thèmes suivants :

Groupe	Ce qui est vérifié	Vulnérabilités
T1	Scan des secrets : plus aucun mot de passe / token / clé codé en dur dans src/ et api/	V1, V3, V4, V5, V6
T2	Endpoint serveur de vérification des mots de passe (/api/verify-access)	V3, V5
T3	Liste blanche des chemins du proxy WordPress	V8
T4	Proxy Apimo : CORS restreint, lecture seule, secrets en variables d'environnement	V1, V2
T5	Chatbot : assainissement des entrées + durcissement anti-injection de prompt	V9
T6	Améliorations globales : en-têtes de sécurité, CORS, endpoint debug, DOMPurify	Toutes

Chaque section ci-dessous indique, pour chaque vulnérabilité : le problème (en langage simple), le correctif appliqué (comment le code a été modifié, avec avant / après), les fichiers modifiés, puis la rubrique « Test de vérification » qui liste les tests automatiques concernés ainsi qu'une vérification manuelle possible.

V1 — Identifiants API Apimo codés en dur Haute

LE PROBLÈME

Le token de l'API Apimo (le CRM des annonces immobilières) ainsi que l'ID du fournisseur étaient écrits en clair dans le JavaScript du navigateur. N'importe quel visiteur pouvait ouvrir « code source de la page » ou télécharger le bundle de production et récupérer le token, puis faire des requêtes authentifiées au CRM Apimo.

AVANT

```
// src/services/apimoService.ts
const APIMO_CONFIG = {
  baseUrl: '/api/apimo',
  agencyId: '25311',
  providerId: '4567',
  token: 'd07da6e744bb033d1299469f1f6f7334531ec05c', // ⚠️ secret exposé
};
```

APRÈS

```
// src/services/apimoService.ts — plus aucun secret côté client
const APIMO_CONFIG = {
  baseUrl: '/api/apimo',
  agencyId: '25311',
};
```

Comment le correctif a été fait :

- `providerId` et `token` ont été **retirés de la config client** (`src/services/apimoService.ts`).
- La méthode `getAuthHeader()` (qui construisait l'en-tête d'authentification côté navigateur) a été supprimée — l'authentification se fait désormais uniquement côté serveur.
- Le proxy `api/apimo.js` lit les identifiants dans les variables d'environnement `APIMO_PROVIDER_ID` et `APIMO_TOKEN` ; s'ils sont absents, il renvoie une erreur de configuration (et ne transmet **aucun** secret).
- Le proxy de développement (`src/setupProxy.js`) a également été mis à jour pour lire `process.env` au lieu de valeurs codées en dur.

VARIABLES D'ENVIRONNEMENT REQUISES

```
APIMO_PROVIDER_ID=<votre_id_fournisseur>
APIMO_TOKEN=<votre_token_api>
```

Tests automatiques (T1 + T4) :

- ✓ Aucun mot de passe/token connu ne figure plus dans le code (`src/` + `api/`)
- ✓ `apimoService.ts` côté client ne contient plus `providerId` ni `token`
- ✓ `api/apimo.js` lit les identifiants depuis les variables d'environnement
- ✓ Identifiants absents → réponse d'erreur sans secret

Vérification manuelle :

```
# Aucun token ne doit apparaître dans le code client :
grep -rn "d07da6e744bb033d1299469f1f6f7334531ec05c" src/ api/ || echo "OK : aucun secret"

# Le bundle de production ne doit contenir aucun identifiant Apimo :
grep -o "APIMO_\|providerId\|25677\|4567" build/static/js/*.js | head || echo "OK : rien exposé"
```

V2 — Proxy API ouvert sans authentification

Critique

LE PROBLÈME

La fonction serverless `/api/apimo` acceptait des requêtes depuis **n'importe quelle origine** (en-tête `Access-Control-Allow-Origin: *`) et sans aucune authentification. Comme elle utilisait les identifiants serveur pour interroger le CRM, n'importe qui pouvait utiliser le site SquareMeter comme **proxy ouvert** et lire l'intégralité du CRM (81 biens, mais aussi les données personnelles des agents : e-mails, téléphones, dates de naissance).

Comment le correctif a été fait (dans `api/apimo.js`) :

- CORS restreint de `*` vers une liste blanche : `https://www.squaremeter.ma` et `https://squaremeter.ma` uniquement.
- Méthodes HTTP limitées à **GET** uniquement (`POST` et `PUT` supprimés) — le proxy est désormais en lecture seule.
- Le transfert du corps de requête a été supprimé (sans objet pour du GET).
- Les logs verbeux qui exposaient des détails de requête ont été nettoyés.

AVANT

```
res.setHeader('Access-Control-Allow-Origin', '*'); // ⚠ ouvert à tous
```

APRÈS

```
const ALLOWED_ORIGINS = ['https://www.squaremeter.ma', 'https://squaremeter.ma'];
// ...
if (ALLOWED_ORIGINS.includes(origin)) {
  res.setHeader('Access-Control-Allow-Origin', origin); // ✅ origine vérifiée
}
res.setHeader('Access-Control-Allow-Methods', 'GET,OPTIONS');
// GET uniquement :
if (req.method !== 'GET') return res.status(405).json({ error: 'Method not allowed' });
```

Tests automatiques (T4) :

- ✓ Origine autorisée → en-tête CORS présent
- ✓ Origine inconnue → aucun en-tête CORS
- ✓ Méthode POST → 405 (lecture seule)
- ✓ Méthode PUT → 405 (lecture seule)

Vérification manuelle :

```
# Une origine inconnue ne doit JAMAIS recevoir l'en-tête CORS :
curl -s -i -H "Origin: https://evil.example.com" \
  "https://squaremeter.ma/api/apimo/agencies/25311/properties?status=1" | grep -i access-control

# Toute méthode autre que GET doit renvoyer 405 :
curl -s -o /dev/null -w "%{http_code}\n" -X POST \
  "https://squaremeter.ma/api/apimo/agencies/25311/properties"
```

LE PROBLÈME

Le mot de passe de l'espace réservé aux clients était écrit en clair dans le composant `ClientsProtectedRoute.tsx` et embarqué dans le JavaScript du navigateur. Le verrouillage anti force-brute était donc inutile : **n'importe qui pouvait lire le mot de passe directement dans le code.**

Comment le correctif a été fait :

- Le mot de passe a été **entièrement retiré du code client.**
- Un nouveau endpoint serveur a été créé : `api/verify-access.js` . Le client envoie le mot de passe à `/api/verify-access` avec `section: 'clients'` .
- Le serveur compare en **temps constant** (`crypto.timingSafeEqual`) pour éviter les attaques temporelles.
- Limitation côté serveur : **5 tentatives maximum**, verrouillage de **1 minute**.
- Le mot de passe est stocké dans la variable d'environnement `CLIENTS_ACCESS_PASSWORD` .

VARIABLES D'ENVIRONNEMENT REQUISES

```
CLIENTS_ACCESS_PASSWORD=<votre_mot_de_passe_clients>
```

Tests automatiques (T1 + T2) :

- ✓ Aucun mot de passe connu ne figure plus dans le code
- ✓ Mot de passe client correct → 200 { authenticated: true }
- ✓ Mauvais mot de passe → 401 avec tentatives restantes
- ✓ 5 échecs consécutifs → 429 (verrouillage anti force brute)
- ✓ Section invalide → 400
- ✓ Mot de passe manquant / non string / trop court → 400
- ✓ Variable d'environnement absente → 500 (refus de servir)
- ✓ Comparaison en temps constant (`crypto.timingSafeEqual`) utilisée

Vérification manuelle :

```
# Mauvais mot de passe → 401, puis après 5 essais → 429 :
curl -s -X POST -H "Content-Type: application/json" \
-d '{"password":"incorrect","section":"clients"}' \
https://squaremeter.ma/api/verify-access -w "\nHTTP %{http_code}\n"
```

V4 — Mot de passe section Mag codé en dur

Faible

LE PROBLÈME

Le composant `MagProtectedRoute.tsx` contenait un mot de passe en clair (`SquareMeter#2025!Mag`) comparé avec un simple `===` , sans protection en temps constant, sans limitation de tentatives et sans verrouillage.

Comment le correctif a été fait :

- Une analyse du projet a montré que ce composant **n'était importé nulle part** et donc totalement inutilisé.

- Le fichier a été **supprimé** (`src/components/ProtectedRoute/MagProtectedRoute.tsx`).

Test automatique (T1) :

✓ `MagProtectedRoute.tsx` (V4) a bien été supprimé

Vérification manuelle :

```
ls src/components/ProtectedRoute/  
# Doit afficher : AuthProtectedRoute.tsx  ClientsProtectedRoute.tsx  PropertyStatsProtectedRoute.tsx  
# MagProtectedRoute.tsx ne doit plus exister.
```

V5 — Mot de passe du tableau de bord analytique codé en dur Critique

LE PROBLÈME

Le mot de passe du tableau de bord des statistiques immobilières était codé en dur dans `PropertyStatsProtectedRoute.tsx` et livré dans le bundle navigateur. Comme pour V3, le mécanisme de verrouillage était inutile car **le mot de passe était publiquement visible**.

Comment le correctif a été fait :

- Le mot de passe a été retiré du code client.
- Le client envoie maintenant le mot de passe au même endpoint `/api/verify-access` avec `section: 'analytics'`.
- Vérification côté serveur : comparaison en temps constant + limitation de tentatives.
- Le mot de passe est stocké dans `ANALYTICS_ACCESS_PASSWORD`.

VARIABLE D'ENVIRONNEMENT REQUISE

```
ANALYTICS_ACCESS_PASSWORD=<votre_mot_de_passe_analytics>
```

Tests automatiques (T1 + T2) :

- ✓ Aucun mot de passe connu ne figure plus dans le code
- ✓ Mot de passe analytics correct → 200 { authenticated: true }
- ✓ Mauvais mot de passe → 401 avec tentatives restantes
- ✓ 5 échecs consécutifs → 429 (verrouillage anti force brute)
- ✓ Origine non autorisée → aucun en-tête Access-Control-Allow-Origin

Vérification manuelle :

```
# Le bundle JavaScript de production ne doit contenir aucun des mots de passe historiques :  
grep -r1 "SM-TEAM::Analytics\|SM-TEAM::Clients" build/ || echo "OK : rien exposé"
```

V6 — Clé IA Gemini exposée au navigateur Moyenne

LE PROBLÈME

La clé API Google Gemini était stockée dans `REACT_APP_GEMINI_API_KEY`. React embarque automatiquement toute variable préfixée `REACT_APP_` dans le JavaScript livré au navigateur. La clé était donc visible par tous et utilisée directement depuis le navigateur par le service chatbot (`ragChatbotService.ts`) et le service d'embeddings (`embeddingService.ts`).

Comment le correctif a été fait :

- Dans `api/chatbot.js`, la variable a été renommée en `GEMINI_API_KEY` (sans le préfixe `REACT_APP_`) : elle n'est donc **plus embarquée dans le bundle client**.
- `ragChatbotService.ts` a été réécrit : tous les appels IA passent désormais par le endpoint serveur `/api/chatbot`.
- L'import `GoogleGenerativeAI` et l'utilisation directe du modèle ont été retirés du service client.
- Les champs `genAI` et `model` ont été supprimés de la classe du chatbot.

AVANT

```
const key = apiKey || process.env.REACT_APP_GEMINI_API_KEY; // ⚠️ exposé au navigateur
this.genAI = new GoogleGenerativeAI(key);
this.model = this.genAI.getGenerativeModel({ model: 'gemini-2.5-flash' });
```

APRÈS

```
// api/chatbot.js – clé lue uniquement côté serveur :
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY || '');

// src/services/ragChatbotService.ts – le client appelle le proxy :
const res = await fetch('/api/chatbot', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ message, language }),
});
```

VARIABLE D'ENVIRONNEMENT REQUISE

```
GEMINI_API_KEY=<votre_cle_gemini>
```

POINT D'ATTENTION

`src/services/embeddingService.ts` référence encore `REACT_APP_GEMINI_API_KEY` pour les embeddings de texte. Ce service doit être migré vers un endpoint serveur dans une future mise à jour (action notée dans `SECURITY_FIXES.md`). Il n'est pas utilisé par le chatbot en production.

Tests automatiques (T1 + T6) :

- ✓ Le chatbot client ne contient plus de clé Gemini côté navigateur
- ✓ `api/chatbot.js` utilise `GEMINI_API_KEY` (côté serveur uniquement)
- ✓ Aucun appel direct à l'API Gemini dans le bundle client
- ✓ V6 — `api/chatbot.js` lit la clé Gemini côté serveur (pas `REACT_APP`)

Vérification manuelle :

```
# Aucune clé ne doit apparaître dans le bundle de production :  
grep -roE "AIza[0-9A-Za-z_-]{20,}" build/ | head || echo "OK : aucune clé Gemini exposée"
```

V7 — XSS stocké via dangerouslySetInnerHTML Haute

LE PROBLÈME

Le contenu des articles WordPress était récupéré via le proxy puis affiché avec `dangerouslySetInnerHTML` **sans aucun assainissement**. Le contenu WordPress peut contenir des balises `<script>`, `<iframe>`, `<object>` ou des attributs de gestionnaire d'événements (`onerror`, `onload`), ce qui permettait une **attaque XSS stockée** (exécution de code malveillant dans le navigateur des visiteurs).

Comment le correctif a été fait :

- Installation de `dompurify` (et de `@types/dompurify` pour TypeScript).
- Ajout d'une fonction `sanitizeHtml()` dans `MagArticle.tsx` qui applique DOMPurify avec une **liste blanche explicite** de balises et d'attributs sûrs.
- Tout le HTML WordPress est assaini **avant** d'être placé dans `dangerouslySetInnerHTML`.
- Balises bloquées : `<script>`, `<iframe>`, `<object>`, `<embed>`, `<form>`, `<input>`, `<svg>`, `<math>`, etc.
- Attributs bloqués : `onclick`, `onerror`, `onload`, `onmouseover`, etc.

AVANT

```
<div dangerouslySetInnerHTML={{ __html: processedContent }} />
```

APRÈS

```
function sanitizeHtml(html) {  
  return DOMPurify.sanitize(html, {  
    ALLOWED_TAGS: ['p','br','strong','em','a','img','h2','h3','ul','ol','li', ...],  
    ALLOWED_ATTR: ['href','src','alt','title', ...],  
    ALLOW_DATA_ATTR: false,  
  });  
}  
// ... puis toutes les écritures passent par sanitizeHtml :  
setProcessedContent(sanitizeHtml(c));
```

Test automatique (T6) :

✓ V7 — Le contenu WordPress est assaini par DOMPurify avant affichage

Vérification manuelle :

```
# Toute balise dangereuse insérée dans un article doit être neutralisée.  
# Dans le navigateur, ouvrir un article et inspecter le HTML rendu :  
# aucune balise
```